

The vox Ratchet Tools: A Z Specification

Formal model of check-oo, check-coupling, check-suppressions

July 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Metric Ordering	2
5	Metric Maps	3
6	The Shared Fail-Closed Core	4
6.1	The git environment	4
6.2	Base unresolvable	4
6.3	Base resolves but carries no baseline blob	5
6.4	Base baseline blob corrupt	5
6.5	Reaching the comparison	5
6.6	The CI-write guard	5
6.7	Fail-closed properties	6
7	check-oo: The Must-Improve Ratchet	6
7.1	Inputs	6
8	check-coupling: The Regression-Only Ratchet	8
8.1	The empty-baseline intercept	8
8.2	The comparison	8
9	check-suppressions: The Scalar-Count Ratchet	8
10	The Never-Loosen Update	9
10.1	The update operation	9
10.2	The never-loosen property	10
11	The Suppression Update: A Divergence	10
12	Findings	11
12.1	The OO ratchet does not harden its base baseline	11
12.2	The OO ratchet has no empty-baseline intercept	11
12.3	Suppression update is not never-loosen	11
13	System Invariants	11

1 Introduction

Vox enforces three code-quality ratchets, each a command-line gate run under `make check` and in CI:

- **check-oo** (`tools/oo_ratchet`): a *must-improve* ratchet on per-file object-orientation metrics.
- **check-coupling** (`tools/coupling`): a *regression-only* ratchet on per-file coupling/cohesion metrics.
- **check-suppressions** (`tools/suppression`): a *regression-only* ratchet on a single scalar — the total count of lint/type suppression comments plus **per-file-ignores**.

PR #312 unified all three onto one *fail-closed*, *base-commit-authoritative* core. They differ only in what they compare. This document therefore models the shared core once — base resolution, the fail-closed transition set, the CI-write guard, and the never-loosen update contract — and then adds one operation schema per tool for the part that genuinely differs.

The specification type-checks under `fuzz -t`. Conjectures (written `\vdash?`) record the load-bearing properties; `fuzz` checks that they are well-typed. Two of them (sections 11) record where the code diverges from the informal brief; they are stated as *existence* conjectures because they document behaviour the brief did not predict.

2 Basic Types

The three carriers the model needs no internal structure for: repository relative Python file paths, metric names, and git commit hashes.

$$[FILE, METRIC, COMMIT]$$

3 Free Types

The Boolean type: `fuzz` has none built in, and `ZBOOL` avoids the `BOOL/true/false` keyword clash that would defeat `ProB`.

$$ZBOOL ::= ztrue \mid zfalse$$

A metric's ordering direction. `dGeq` means larger is better ($\geq$ threshold), `dLeq` smaller is better ($\leq$), and `dEq` means closer to a fixed target is better ($=$).

$$Direction ::= dGeq \mid dLeq \mid dEq$$

The four outcomes a ratchet operation can produce. `pass` and `bootstrapPass` exit zero; `fail` and `failClosedError` exit non-zero. `bootstrapPass` is reserved for a genuine first adoption (no baseline anywhere); `failClosedError` is a controlled non-zero raised when git or the baseline blob is broken.

$$OUTCOME ::= pass \mid bootstrapPass \mid fail \mid failClosedError$$
$$\frac{}{exitCode : OUTCOME \rightarrow \mathbb{N}} \quad exitCode = \{pass \mapsto 0, bootstrapPass \mapsto 0, fail \mapsto 1, failClosedError \mapsto 1\}$$

The fail-closed characterisation of the exit code is fixed by the definition above: `exitCode o = 0` exactly when $o = pass$ or $o = bootstrapPass$. The two non-zero outcomes are the two failure kinds.

4 Metric Ordering

Each metric carries a direction and an absolute target. These are the `Thresholds` tables in the code (`oo_ratchet/thresholds.py`, `coupling/thresholds.py`). The model leaves the tables abstract: only the ordering they induce matters to the ratchet.

$$\begin{array}{l} \text{metricDir} : \text{METRIC} \rightarrow \text{Direction} \\ \text{metricTarget} : \text{METRIC} \rightarrow \mathbb{Z} \end{array}$$

Metric values are integers here. The code uses floats; **fuzz** has no reals, and the ratchet depends only on the *ordering* of values, never on their exact magnitude, so $\mathbb{Z}$ carries the model faithfully.

The absolute value, defined without unary minus so **fuzz** and ProB stay happy.

$$\begin{array}{l} \text{zabs} : \mathbb{Z} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{Z} \bullet (n \geq 0 \Rightarrow \text{zabs } n = n) \\ \quad \wedge (n < 0 \Rightarrow \text{zabs } n + n = 0) \end{array}$$

The distance of a value from a metric’s target (used only by the **dEq** direction).

$$\begin{array}{l} \text{dist} : \text{METRIC} \times \mathbb{Z} \rightarrow \mathbb{N} \\ \hline \forall m : \text{METRIC}; v : \mathbb{Z} \bullet \text{dist}(m, v) = \text{zabs}(v - \text{metricTarget } m) \end{array}$$

“At least as good as”: the ratchet’s non-regression test.

$$\begin{array}{l} \text{betterEq} : \text{METRIC} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; c, b : \mathbb{Z} \bullet \text{betterEq}(m, c, b) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge c \geq b) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge c \leq b) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge \text{dist}(m, c) \leq \text{dist}(m, b)) \end{array}$$

“Strictly better”: the must-improve test.

$$\begin{array}{l} \text{strictBetter} : \text{METRIC} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; c, b : \mathbb{Z} \bullet \text{strictBetter}(m, c, b) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge c > b) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge c < b) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge \text{dist}(m, c) < \text{dist}(m, b)) \end{array}$$

“Meets its absolute threshold”: used to grade a brand-new file that has no baseline to improve against.

$$\begin{array}{l} \text{meets} : \text{METRIC} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; v : \mathbb{Z} \bullet \text{meets}(m, v) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge v \geq \text{metricTarget } m) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge v \leq \text{metricTarget } m) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge v = \text{metricTarget } m) \end{array}$$

5 Metric Maps

A file’s metrics are a partial map from metric name to value; a baseline is a partial map from file to metric map.

$$\text{MMAP} == \text{METRIC} \rightarrow \mathbb{Z}$$

$$\text{BLINE} == \text{FILE} \rightarrow \text{MMAP}$$

The metrics of **c** that are worse than **b** — the regressed set. A metric absent from **b** never regresses (a new metric is not a regression).

$$\begin{array}{l} \text{worse} : \text{MMAP} \times \text{MMAP} \rightarrow \text{F METRIC} \\ \hline \forall c, b : \text{MMAP} \bullet \text{worse}(c, b) = \\ \quad \{m : \text{dom } c \mid m \in \text{dom } b \wedge \text{betterEq}(m, c \ m, b \ m) = \text{zfalse}\} \end{array}$$

The metrics of **c** that strictly improve on **b**.

$better : MMAP \times MMAP \rightarrow \mathbb{F} \text{ METRIC}$
$\forall c, b : MMAP \bullet better(c, b) =$ $\{m : \text{dom } c \mid m \in \text{dom } b \wedge strictBetter(m, c\ m, b\ m) = ztrue\}$

Whether every metric of a map clears its absolute threshold.

$allMeet : MMAP \rightarrow ZBOOL$
$\forall c : MMAP \bullet allMeet\ c = ztrue \Leftrightarrow$ $(\forall m : \text{dom } c \bullet meets(m, c\ m) = ztrue)$

6 The Shared Fail-Closed Core

Every check resolves a comparison base, reads the baseline committed *at that base commit*, and only then compares. The resolution and the fail-closed decisions that precede the comparison are identical across all three tools (`ratchet.py` for check-oo, `ratchet.py` for check-coupling, `baseline.py` for check-suppressions).

6.1 The git environment

`GitEnv` records what the resolution phase can observe: whether the base resolves, whether a baseline blob exists at the base and whether it is corrupt, the same for the `origin/main` tip, whether an in-tree baseline file is present, and whether `--require-base` was passed. The invariants encode the obvious well-formedness: a blob can be corrupt only if it exists, and can exist only if its ref resolves.

<i>GitEnv</i>
$baseResolves : ZBOOL$
$baseHasBlob : ZBOOL$
$baseCorrupt : ZBOOL$
$tipResolves : ZBOOL$
$tipHasBlob : ZBOOL$
$tipCorrupt : ZBOOL$
$inTreeExists : ZBOOL$
$requireBase : ZBOOL$
$baseCorrupt = ztrue \Rightarrow baseHasBlob = ztrue$
$baseHasBlob = ztrue \Rightarrow baseResolves = ztrue$
$tipCorrupt = ztrue \Rightarrow tipHasBlob = ztrue$
$tipHasBlob = ztrue \Rightarrow tipResolves = ztrue$

6.2 Base unresolvable

When no base commit can be found (`_no_base`): under `--require-base` the gate fails; a genuine first adoption — no in-tree baseline to protect — bootstraps; otherwise an in-tree baseline with an unresolvable base means a stale or unfetched `origin/main`, and the gate fails loudly rather than pass on trust.

<i>NoBase</i>
<i>GitEnv</i>
$out! : OUTCOME$
$baseResolves = zfalse$
$requireBase = ztrue \Rightarrow out! = fail$
$requireBase = zfalse \wedge inTreeExists = zfalse \Rightarrow$ $out! = bootstrapPass$
$requireBase = zfalse \wedge inTreeExists = ztrue \Rightarrow out! = fail$

6.3 Base resolves but carries no baseline blob

When the base commit has no baseline (`_absent_base_baseline`): a genuine first adoption requires the `origin/main` tip to also lack a baseline. A corrupt tip blob is a controlled error. A tip that carries a baseline means the branch forked before adoption and would launder a regression past the empty base, so it fails. An unresolvable tip with an in-tree baseline present cannot confirm first adoption, so it fails closed.

<i>AbsentBase</i> <i>GitEnv</i> <i>out!</i> : <i>OUTCOME</i>
$baseResolves = ztrue \wedge baseHasBlob = zfalse$ $tipCorrupt = ztrue \Rightarrow out! = failClosedError$ $tipResolves = zfalse \wedge inTreeExists = ztrue \Rightarrow out! = fail$ $tipResolves = zfalse \wedge inTreeExists = zfalse \Rightarrow$ $out! = bootstrapPass$ $tipResolves = ztrue \wedge tipCorrupt = zfalse \wedge tipHasBlob = ztrue$ $\Rightarrow out! = fail$ $tipResolves = ztrue \wedge tipHasBlob = zfalse \Rightarrow$ $out! = bootstrapPass$

6.4 Base baseline blob corrupt

A base baseline that resolves and exists but fails to parse (or, for coupling and suppression, is non-dict / non-numeric / non-UTF8) raises `GitError`, which the CLI turns into a controlled non-zero.

<i>BaseError</i> <i>GitEnv</i> <i>out!</i> : <i>OUTCOME</i>
$baseResolves = ztrue \wedge baseHasBlob = ztrue \wedge baseCorrupt = ztrue$ $out! = failClosedError$

6.5 Reaching the comparison

The comparison runs only when the base resolves and carries a readable baseline blob. `Proceed` names that precondition; each per-tool check schema includes it.

<i>Proceed</i> <i>GitEnv</i>
$baseResolves = ztrue \wedge baseHasBlob = ztrue \wedge baseCorrupt = zfalse$

6.6 The CI-write guard

Every mutation (update, relax, reconcile, rebase) refuses to write under `GITHUB_ACTIONS` unless `--allow-ci-write` is passed. The guard is the same in all three tools.

<i>CiGuard</i> $githubActions : ZBOOL$ $allowCiWrite : ZBOOL$ $blocked! : ZBOOL$
$blocked! = ztrue \Leftrightarrow$ $(githubActions = ztrue \wedge allowCiWrite = zfalse)$

6.7 Fail-closed properties

The fail-closed contract is that `bootstrapPass` arises *only* from a genuine absence of baseline. These property schemas restate the guarantee as type-checked predicates over the transition schemas; each is a redundant strengthening of the schema it includes, so it has exactly the same models — it documents the property without weakening it.

<i>NoBaseFailClosed</i>	_____
<i>NoBase</i>	_____
$out! = bootstrapPass \Rightarrow inTreeExists = zfalse$	_____

<i>AbsentBaseFailClosed</i>	_____
<i>AbsentBase</i>	_____
$out! = bootstrapPass \Rightarrow$ $(inTreeExists = zfalse$ $\vee (tipResolves = ztrue \wedge tipHasBlob = zfalse))$	_____

7 check-oo: The Must-Improve Ratchet

The OO ratchet is the strict one. On the comparison path it requires, of a change that touches any file already tracked in the base baseline, that at least one metric *strictly* improve on such a file, and that no metric regress — unless a regression is waived by an audited relaxation.

7.1 Inputs

`OoCheck` carries the resolved base commit and the base-commit baseline (`baselineAt baseCommit`, the authoritative comparison source — *never* the in-tree file); the in-tree baseline (read only for the lock-in and waiver checks); the current scores; the touched set (touched Python files that scored clean); the touched files that failed to parse; and the waivable (`file, metric`) pairs harvested from audit relaxations recorded since the base commit.

OoCheck

Proceed

baseCommit : *COMMIT*

baselineAt : *COMMIT* $\rightarrow$ *BLINE*

baseB : *BLINE*

inTree : *BLINE*

current : *BLINE*

touched : $\mathbb{F}$ *FILE*

brokenTouched : $\mathbb{F}$ *FILE*

waivable : $\mathbb{F}$ (*FILE* $\times$ *METRIC*)

wvd : $\mathbb{F}$ (*FILE* $\times$ *METRIC*)

rawRegressed : $\mathbb{F}$ (*FILE* $\times$ *METRIC*)

regressedP : $\mathbb{F}$ (*FILE* $\times$ *METRIC*)

waivedP : $\mathbb{F}$ (*FILE* $\times$ *METRIC*)

missingF : $\mathbb{F}$ *FILE*

lockFailF : $\mathbb{F}$ *FILE*

basePresent : $\mathbb{F}$ *FILE*

improvedExisting : *ZBOOL*

newPasses : *ZBOOL*

improvementOk : *ZBOOL*

out! : *OUTCOME*

baseCommit $\in \text{dom } \text{baselineAt}$

baseB = *baselineAt* *baseCommit*

touched $\subseteq \text{dom } \text{current}$

touched $\cap$ *brokenTouched* = $\emptyset$

wvd = $\{f : \text{touched}; m : \text{METRIC} \mid f \in \text{dom } \text{inTree} \wedge m \in \text{dom}(\text{inTree } f) \wedge \text{inTree } f \text{ } m = \text{current } f \text{ } m \wedge (f, m) \in$

rawRegressed = $\{f : \text{touched}; m : \text{METRIC} \mid f \in \text{dom } \text{baseB} \wedge m \in \text{worse}(\text{current } f, \text{baseB } f)\}$

regressedP = *rawRegressed* $\setminus$ *wvd*

waivedP = *rawRegressed* $\cap$ *wvd*

missingF = $\{f : \text{touched} \mid f \notin \text{dom } \text{inTree}\}$

lockFailF = $\{f : \text{touched} \mid f \in \text{dom } \text{inTree} \wedge (\exists m : \text{dom}(\text{current } f) \bullet m \notin \text{dom}(\text{inTree } f) \vee \text{inTree } f \text{ } m \neq \text{current } f \text{ } m)$

basePresent = *touched* $\cap \text{dom } \text{baseB}$

improvedExisting = *ztrue* $\Leftrightarrow$

$(\exists f : \text{basePresent} \bullet \text{better}(\text{current } f, \text{baseB } f) \neq \emptyset)$

newPasses = *ztrue* $\Leftrightarrow$

$(\exists f : \text{touched} \setminus \text{dom } \text{baseB} \bullet \text{allMeet}(\text{current } f) = \text{ztrue})$

improvementOk = *ztrue* $\Leftrightarrow$

waivedP $\neq \emptyset$

$\vee (\text{basePresent} \neq \emptyset \wedge \text{improvedExisting} = \text{ztrue})$

$\vee (\text{basePresent} = \emptyset \wedge \text{newPasses} = \text{ztrue})$

brokenTouched $\neq \emptyset \Rightarrow \text{out!} = \text{fail}$

brokenTouched = $\emptyset \wedge \text{touched} = \emptyset \Rightarrow \text{out!} = \text{pass}$

brokenTouched = $\emptyset \wedge \text{touched} \neq \emptyset \Rightarrow$

$(\text{out!} = \text{pass} \Leftrightarrow$

$(\text{missingF} = \emptyset \wedge \text{lockFailF} = \emptyset$

$\wedge \text{regressedP} = \emptyset \wedge \text{improvementOk} = \text{ztrue}))$

out! $\neq \text{bootstrapPass} \wedge \text{out!} \neq \text{failClosedError}$

The waiver is deliberately narrow: a metric is waived (**RELAXED**) only when the in-tree baseline has locked the current value (so the regression was recorded, not smuggled) *and* the audit log carries a matching relaxation. The improvement gate follows the code's S5: if any touched file is tracked in the base, the required improvement must come from such a file; a pure-add change is satisfied by a new file clearing all absolute thresholds; a change carrying a waiver is exempt.

8 check-coupling: The Regression-Only Ratchet

The coupling ratchet drops the must-improve gate, the waiver, the lock-in check, and the completeness check. A touched file fails only if a metric is strictly worse than its base-commit baseline; holding steady passes.

8.1 The empty-baseline intercept

Coupling adds one guard the OO ratchet lacks. An empty $\{\}$ base baseline (a truncated write or a bad merge) under `--require-base` fails closed before the comparison: otherwise every touched file would look new and no regression could be seen.

<i>CplEmptyRequire</i> <i>Proceed</i> $baseCommit : COMMIT$ $baselineAt : COMMIT \rightarrow BLINE$ $baseB : BLINE$ $out! : OUTCOME$
$baseCommit \in \text{dom } baselineAt$ $baseB = baselineAt \ baseCommit$ $baseB = \emptyset$ $requireBase = ztrue$ $out! = fail$

8.2 The comparison

The comparison runs when the base baseline is non-empty, or empty without `--require-base` (in which case every file is new and passes). A file absent from the base is new: it never regresses.

<i>CplCheck</i> <i>Proceed</i> $baseCommit : COMMIT$ $baselineAt : COMMIT \rightarrow BLINE$ $baseB : BLINE$ $current : BLINE$ $touched : \mathbb{F} \text{ FILE}$ $brokenTouched : \mathbb{F} \text{ FILE}$ $regressedP : \mathbb{F}(\text{FILE} \times \text{METRIC})$ $out! : OUTCOME$
$baseCommit \in \text{dom } baselineAt$ $baseB = baselineAt \ baseCommit$ $\neg (baseB = \emptyset \wedge requireBase = ztrue)$ $touched \subseteq \text{dom } current$ $touched \cap brokenTouched = \emptyset$ $regressedP = \{f : touched; m : METRIC \mid f \in \text{dom } baseB \wedge m \in worse(current \ f, baseB \ f)\}$ $brokenTouched \neq \emptyset \Rightarrow out! = fail$ $brokenTouched = \emptyset \wedge touched = \emptyset \Rightarrow out! = pass$ $brokenTouched = \emptyset \wedge touched \neq \emptyset \Rightarrow$ $(out! = pass \Leftrightarrow regressedP = \emptyset)$ $out! \neq bootstrapPass \wedge out! \neq failClosedError$

9 check-suppressions: The Scalar-Count Ratchet

The suppression ratchet compares a single scalar: the total count of `noqa`, `type: ignore`, `pylint: disable`, `pyright: ignore` comments on code lines, plus the number of `ruff per-file-ignores` rule codes. The check fails only if the current total exceeds the base-commit total. The base total is coerced

defensively (`_as_int`): a non-numeric, boolean, NaN, or infinite value becomes 0, which fails closed — a smaller base makes the current total register as an increase.

$ \begin{array}{l} \textit{SupCheck} \\ \textit{Proceed} \\ \textit{baseCommit} : \textit{COMMIT} \\ \textit{supBaselineAt} : \textit{COMMIT} \rightarrow \mathbb{N} \\ \textit{baseTotal} : \mathbb{N} \\ \textit{curTotal} : \mathbb{N} \\ \textit{out!} : \textit{OUTCOME} \end{array} $
$ \begin{array}{l} \textit{baseCommit} \in \text{dom } \textit{supBaselineAt} \\ \textit{baseTotal} = \textit{supBaselineAt } \textit{baseCommit} \\ \textit{curTotal} > \textit{baseTotal} \Rightarrow \textit{out!} = \textit{fail} \\ \textit{curTotal} \leq \textit{baseTotal} \Rightarrow \textit{out!} = \textit{pass} \\ \textit{out!} \neq \textit{bootstrapPass} \wedge \textit{out!} \neq \textit{failClosedError} \end{array} $

10 The Never-Loosen Update

`--update` tightens a baseline but never loosens it. The base ref scopes the touched set only; the regression test is against the *in-tree* baseline. `check-oo` and `check-coupling` share the algorithm (`apply.py`, `writer.py`); `check-suppressions` does not (see Findings).

The rename-aware base lookup: a file inherits its predecessor’s entry when it is a rename target absent from the baseline under its new name, so a regression cannot launder itself through a rename. A file with no entry and no rename predecessor is new; the empty metric map models “no entry” (its `worse` set is empty, so a new file never regresses).

$ \textit{baseFor} : \textit{BLINE} \times (\textit{FILE} \rightarrow \textit{FILE}) \times \textit{FILE} \rightarrow \textit{MMAP} $
$ \begin{array}{l} \forall b : \textit{BLINE}; r : \textit{FILE} \rightarrow \textit{FILE}; f : \textit{FILE} \bullet \\ (f \in \text{dom } b \Rightarrow \textit{baseFor}(b, r, f) = b f) \\ \wedge (f \notin \text{dom } b \wedge f \in \text{dom } r \wedge r f \in \text{dom } b \Rightarrow \textit{baseFor}(b, r, f) = b (r f)) \\ \wedge (f \notin \text{dom } b \wedge \neg (f \in \text{dom } r \wedge r f \in \text{dom } b) \Rightarrow \textit{baseFor}(b, r, f) = \emptyset) \end{array} $

10.1 The update operation

The in-tree baseline is the state. A touched file is *writable* when it scored clean and does not regress against its rename-aware in-tree base; *refused* when it scored clean but regresses (its old entry is kept, so the carried base survives to the next run); dropped when it is gone from disk. Writing a rename target drops the stale source key. The write refuses under the CI guard, and fails the operation if anything was refused or unparseable.

NeverLoosenUpdate

inTree : *BLINE*
inTree' : *BLINE*
current : *BLINE*
touched : $\mathbb{F}$ *FILE*
renames : *FILE* $\bowtie$ *FILE*
parseErrTouched : $\mathbb{F}$ *FILE*
githubActions : *ZBOOL*
allowCiWrite : *ZBOOL*
guarded : *ZBOOL*
writable : $\mathbb{F}$ *FILE*
refusedF : $\mathbb{F}$ *FILE*
droppedMiss : $\mathbb{F}$ *FILE*
droppedSet : $\mathbb{F}$ *FILE*
updates : *BLINE*
out! : *OUTCOME*

guarded = *ztrue* $\Leftrightarrow$ (*githubActions* = *ztrue* $\wedge$ *allowCiWrite* = *zfalse*)
writable = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \in \text{dom } \textit{current} \wedge \textit{worse}(\textit{current } f, \textit{baseFor}(\textit{inTree}, \textit{renames}, f))\}$
refusedF = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \in \text{dom } \textit{current} \wedge \textit{worse}(\textit{current } f, \textit{baseFor}(\textit{inTree}, \textit{renames}, f))\}$
droppedMiss = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \notin \text{dom } \textit{current}\}$
droppedSet = *droppedMiss* $\cup$ *ran*(*writable* $\triangleleft$ *renames*)
updates = *writable* $\triangleleft$ *current*
guarded = *ztrue* $\Rightarrow$ (*out!* = *fail* $\wedge$ *inTree'* = *inTree*)
guarded = *zfalse* $\Rightarrow$
 inTree' = (*droppedSet* $\triangleleft$ *inTree*) $\oplus$ *updates*
guarded = *zfalse* $\wedge$ *refusedF* = $\emptyset$ $\wedge$ *parseErrTouched* = $\emptyset$
 $\Rightarrow$ *out!* = *pass*
guarded = *zfalse* $\wedge$ (*refusedF* $\neq \emptyset \vee \textit{parseErrTouched} \neq \emptyset$)
 $\Rightarrow$ *out!* = *fail*
out! $\neq$ *bootstrapPass* $\wedge$ *out!* $\neq$ *failClosedError*

10.2 The never-loosen property

The guarantee: no metric of any file present both before and after the update is worse afterwards. This property schema restates it as a type-checked predicate over the operation.

NeverLoosens

NeverLoosenUpdate

$\forall f : \text{dom } \textit{inTree}' \cap \text{dom } \textit{inTree} \bullet$
 worse(*inTree'* *f*, *inTree* *f*) = $\emptyset$

11 The Suppression Update: A Divergence

check-suppressions' **update** does *not* implement never-loosen. After the CI guard it writes the current total unconditionally — it takes no base ref, does no scoping, and refuses no increase. The written total may exceed the prior total.

SupUpdate

supTotal : $\mathbb{N}$

supTotal' : $\mathbb{N}$

curTotal : $\mathbb{N}$

githubActions : *ZBOOL*

allowCiWrite : *ZBOOL*

guarded : *ZBOOL*

out! : *OUTCOME*

$guarded = ztrue \Leftrightarrow (githubActions = ztrue \wedge allowCiWrite = zfalse)$

$guarded = ztrue \Rightarrow (out! = fail \wedge supTotal' = supTotal)$

$guarded = zfalse \Rightarrow (out! = pass \wedge supTotal' = curTotal)$

12 Findings

Modelling the code faithfully surfaced three places where behaviour differs from the informal brief. The model follows the code.

12.1 The OO ratchet does not harden its base baseline

The brief lumped all three tools together: “corrupt / non-dict / non-UTF8 / NaN / bool baseline $\rightarrow$ a controlled fail; none may yield pass.” That holds for check-coupling and check-suppressions, whose blob readers reject a non-dict, non-numeric, boolean, or non-UTF8 baseline as a `GitError` (a controlled non-zero). It does *not* hold for check-oo: `oo_ratchet/gitio.py`’s `show_baseline` catches only `json.JSONDecodeError`. A parseable-but-ill-typed base baseline (a string value where a metric map is expected) is not rejected; the comparison then indexes a string and raises an *uncaught TypeError* — not the controlled *fail* the brief assumes. In the model, `BaseError` covers only the unparseable case for check-oo; the ill-typed-but-parseable case has no controlled outcome.

12.2 The OO ratchet has no empty-baseline intercept

check-coupling fails closed on an empty `{}` base baseline under `--require-base` (`CplEmptyRequire`). check-oo has no such guard: an empty base baseline flows straight into the comparison, where every touched file is treated as new. A pure-add change whose new files all clear their absolute thresholds then *passes* — not as a genuine first-adoption `bootstrapPass`, but as an ordinary `pass`. `OoCheck` admits exactly this: with `baseB` = $\emptyset$, `basePresent` = $\emptyset$, and `newPasses` = `ztrue`, the verdict is `pass`. check-suppressions, by contrast, fails on an empty base because the coerced base total is 0 and any current suppression exceeds it.

12.3 Suppression update is not never-loosen

As above (`SupUpdate`): check-oo and check-coupling refuse any per-metric regression on update; check-suppressions rewrites the total unconditionally once past the CI guard. `SupUpdate` permits `supTotal' > supTotal` with `out! = pass`.

13 System Invariants

The three properties the unified core guarantees, and where each lives in this model:

- **Fail-closed** — `bootstrapPass` arises only from a genuine absence of baseline: `NoBaseFailClosed`, `AbsentBaseFailClosed`, and the `exitCode` definition.
- **Never-loosen** — an update’s written baseline is no worse on any metric: `NeverLoosens` (for check-oo and check-coupling; see Findings for check-suppressions).
- **Base-commit-authoritative** — the comparison reads `baselineAt baseCommit`, never the in-tree baseline: the equality `baseB = baselineAt baseCommit` in `OoCheck`, `CplCheck`, and (as a total) `SupCheck`. The in-tree baseline appears in `OoCheck` only for the lock-in and waiver tests, and in `NeverLoosenUpdate` as the never-loosen target.